

Stochastic Green Edge (SGE): Defensa Adaptativa de APIs en el Borde mediante Modelado Estocástico y Cómputo Verde

Equipo *Stochastic Green Edge* (SGE)

Investigación en Ciberseguridad Cloud y Computación Sostenible

August 8, 2026

Resumen

La inspección estática y exhaustiva del 100% del tráfico entrante hacia una API pública maximiza la detección de amenazas pero impone un consumo energético elevado y constante, en tensión directa con los principios de la computación sostenible. Este artículo presenta **Stochastic Green Edge** (SGE), un mecanismo de defensa perimetral de dos capas —un filtro *Edge* ligero y una función *Serverless* de inspección profunda— que decide de forma adaptativa, petición a petición, cuándo activar la inspección costosa. La decisión se formula como un juego estocástico no cooperativo con información imperfecta entre un Defensor y un Atacante, y se resuelve calculando el equilibrio de Nash en estrategias mixtas [1, 2]. En una simulación de 2,000 peticiones (semilla 7) con una mezcla realista (80% legítimo, 15% bots ruidosos, 5% ataques dirigidos), SGE alcanza una **mitigación global del 94.38%** —bloqueando el 100% de los ataques ligeros y entre $\sim 79\%$ y $\sim 88\%$ de los avanzados— consumiendo un **85.9% menos de energía** que la inspección estática total, lo que se traduce en una **reducción proporcional de la huella de carbono**. La eficiencia proviene de que el 80% del tráfico legítimo permanece en el costo mínimo del Edge ($C_{\text{edge}} = 1$), evitando invocar la capa *Serverless*. La implementación es modular, verificada con *Ruff* y con 13 pruebas en *Pytest*.

Palabras clave: juegos estocásticos, equilibrio de Nash, seguridad de APIs, edge computing, serverless, cómputo verde, eficiencia energética.

1 Introducción

Las APIs públicas reciben tráfico heterogéneo que combina usuarios legítimos, bots automatizados ruidosos y adversarios dirigidos y sigilosos. La práctica de seguridad predominante —someter cada petición a una inspección profunda: sanitización exhaustiva, validación criptográfica y análisis de payload— ofrece máxima cobertura, pero incurre en un gasto de CPU y energía elevado

e *invariante*, con independencia del riesgo real de cada petición. Este sobreprocesamiento tiene un impacto ambiental oculto pero significativo en los centros de datos, cuya reducción es un objetivo central del cómputo verde [4, 3].

La observación que motiva este trabajo es que la mayor parte del tráfico no justifica el costo de la inspección profunda: el tráfico legítimo debe servirse con energía mínima y los bots ruidosos pueden frenarse de forma barata en el borde mediante *rate-limiting*. Únicamente una fracción pequeña y sigilosa —ataques dirigidos que imitan tráfico legítimo— requiere efectivamente la capa costosa. El obstáculo es que esa fracción **no es directamente observable**: el defensor actúa bajo *información imperfecta*.

La computación sin servidor (*serverless*) hace este compromiso aún más explícito: sus funciones efímeras solo consumen recursos cuando se invocan, pero sufren una penalización de *arranque en frío* (*cold start*) cuando se activan tras un periodo de inactividad [5]. Ello convierte la decisión de “inspeccionar o no” en un problema de asignación de recursos con costos asimétricos, natural para un tratamiento por teoría de juegos [1].

Proponemos modelar la decisión de escalado como un juego no cooperativo entre el Defensor y el Atacante y resolver el equilibrio de Nash en estrategias mixtas, obteniendo una política de escalado *aleatorizada* óptima en el sentido de que ninguna de las partes puede mejorar su utilidad esperada desviándose unilateralmente [2]. Nuestras contribuciones son:

- Una formulación analítica de la defensa perimetral como un juego bimatriz 2×2 con una función de utilidad que integra recompensa de seguridad, costo energético y daño esperado.
- Un motor que calcula el equilibrio de Nash en estrategias mixtas y traduce el *principio de indiferencia* en una regla que balancea Joules y seguridad.
- Una evaluación empírica reproducible que cuantifica el compromiso frente a líneas base y explica analíticamente

camente el origen del ahorro energético.

2 Objetivos y Alcance del Proyecto

2.1 Objetivo general

Diseñar, implementar y validar un **motor de decisión estocástico basado en teoría de juegos** que realice la *intercepción eficiente y segura de amenazas* sobre APIs e infraestructuras *Edge*, maximizando la tasa de detección efectiva mientras **minimiza el consumo energético y la huella de carbono** asociados a la inspección de tráfico. El objetivo es ofrecer una alternativa cuantificablemente más sostenible que la inspección profunda continua, sin degradación material de la postura de seguridad.

2.2 Objetivos específicos

1. **(OE-1) Lógica estocástica y de teoría de juegos.** Formalizar la interacción Defensor–Atacante como un juego no cooperativo, derivar el *equilibrio de Nash* en estrategias mixtas y materializar una política de inspección intermitente y adaptativa, reproducible bajo semilla fija (*seed-7*).
2. **(OE-2) Capa de seguridad en APIs.** Construir el módulo de filtrado/inspección de peticiones sobre tráfico HTTP/REST, incluyendo la inspección de *payloads* y *prompts* para la detección de *prompt injection* y abuso de peticiones, y la extracción de *features* de riesgo que retroalimentan a OE-1.
3. **(OE-3) Medición y reducción del consumo energético.** Instrumentar la telemetría de recursos (CPU, memoria, energía por petición) y su conversión a huella de carbono (gCO₂eq), integrando la función de coste energético en el *payoff* del Defensor y cerrando el lazo seguridad–energía.
4. **(OE-4) Validación y benchmarking.** Establecer un marco experimental reproducible que compare SGE frente a métodos convencionales (inspección continua y muestreo uniforme fijo), reportando la relación eficiencia energética *vs.* tasa de detección (frontera de Pareto) bajo *seed-7*.

2.3 Alcance (*scope*)

Esta fase persigue un **prototipo funcional validado experimentalmente**, no un producto listo para producción a gran escala.

Dentro del alcance (*in-scope*).

- Algoritmo estocástico y cálculo del equilibrio de Nash (estrategia mixta) con reproducibilidad garantizada (*seed-7*).
- Módulo de filtrado de peticiones/APIs sobre **HTTP/REST** (proxy/gateway de inspección, *rate-limiting* y aplicación de veredictos).
- Inspección de *payloads/prompts* (*prompt injection* y patrones de abuso) y extracción de señales de riesgo.
- Métricas de consumo energético y su traducción a gCO₂eq, integradas en la función de utilidad.
- Prototipo funcional integrado y suite de *benchmarking* comparativo con salidas reproducibles.

Fuera del alcance (*out-of-scope*) en esta fase.

- Despliegue en clústeres multirregión a gran escala (alta disponibilidad global, *autoscaling* productivo).
- Interfaces gráficas complejas de administración.
- Soporte para protocolos industriales propietarios no HTTP/REST (p. ej. gRPC, WebSocket, MQTT, OPC-UA, Modbus).
- Certificación/*hardening* productivo, *pentesting* formal y cumplimiento regulatorio.

Entregables clave. (i) Artefactos de código —motor de decisión estocástico y proxy/gateway de inspección; (ii) documentación técnica y modelo matemático de la estrategia de inspección; y (iii) reporte de *benchmarks* que contrasta la eficiencia energética frente a la tasa de detección de amenazas.

3 Formulación Matemática y Modelo del Juego

Modelamos cada petición *sospechosa* como una etapa de un juego estocástico no cooperativo entre dos jugadores racionales [1]. Al no observar el defensor la naturaleza real de la petición, el juego es de información imperfecta y, como veremos, carece de equilibrio en estrategias puras, por lo que la solución natural es un equilibrio en estrategias mixtas [2].

3.1 Jugadores, acciones y utilidad

El **Defensor** dispone de dos estrategias puras: D_V (*Verde*, permanecer en la capa Edge, energía mínima) y D_A (*Activa*, escalar a la inspección Serverless). El **Atacante** elige entre A_L (ataque *ligero*, ruidoso y automático) y A_M (ataque *avanzado*, dirigido y sigiloso). El Defensor juega D_V con probabilidad p y D_A con probabilidad $1 - p$; el Atacante juega A_L con probabilidad q y A_M con $1 - q$.

La utilidad del Defensor ante el par de acciones (a, d) se define como

$$U_D(a, d) = \alpha R(a, d) - \beta C_{\text{energy}}(d) - \gamma C_{\text{damage}}(a, d), \quad (1)$$

donde cada término tiene una interpretación precisa:

- $R(a, d)$: *recompensa* por una detección exitosa o por procesar tráfico legítimo con energía mínima.
- $C_{\text{energy}}(d)$: *costo energético* de la acción defensiva, medido en unidades relativas de Joules/ciclos [3]. Siguiendo el modelo de costos serverless [5], fijamos $C_{\text{edge}} = 1$, $C_{\text{exec}} = 15$ (ejecución tibia) y $C_{\text{cold}} = 25$ (arranque en frío).
- $C_{\text{damage}}(a, d)$: *penalización por daño*, severa cuando un ataque avanzado A_M evade la seguridad por permanecer el Defensor en el borde (D_V).
- $\alpha, \beta, \gamma \geq 0$: pesos de ponderación que fijan la importancia relativa de seguridad, energía y daño.

La ecuación (1) expresa formalmente el dilema de diseño: el segundo término penaliza gastar energía, mientras que el tercero penaliza no gastarla cuando habría sido necesario. El equilibrio de Nash resolverá esta tensión de manera óptima.

3.2 Sospecha e información imperfecta

El Defensor no observa la clase real de la petición; solo infiere una *puntuación de sospecha* $s \in [0, 1]$ a partir de métricas ligeras del Edge, con la anomalía de payload como señal dominante (es lo único que la inspección profunda puede resolver):

$$s = 0.15 \phi_{\text{ip}} + 0.15 \phi_{\text{ua}} + 0.70 \phi_{\text{payload}}, \quad (2)$$

con $\phi_{\text{ip}} = 1 - e^{-\text{freq}/6}$ la saturación de la frecuencia de IP. La sospecha modula el riesgo $\rho = s_0 + (1 - s_0)s$ (piso $s_0 = 0.15$) y con él la severidad del daño $C_{\text{damage}} = D_0 \rho$ y la ganancia de brecha del atacante. Así, el juego —y su equilibrio— es *específico de cada petición*, capturando la naturaleza estocástica del entorno [1].

3.3 Matrices de pago

Con los parámetros del prototipo ($\alpha = \beta = \gamma = 1$, $R_{\text{edge}} = 10$, $R_{\text{deep}} = 12$, $D_0 = 140$, ganancia de brecha $G_0 = 200$, costo de ataque avanzado $K = 20$ y penalizaciones de detección 5 y 15), las entradas de la matriz del Defensor $\mathbf{A}$ (filas D_V, D_A ; columnas A_L, A_M) son

$$A_{D_V, A_L} = \alpha R_{\text{edge}} - \beta C_{\text{edge}}, \quad (3)$$

$$A_{D_V, A_M} = -\beta C_{\text{edge}} - \gamma C_{\text{damage}}, \quad (4)$$

$$A_{D_A, \cdot} = \alpha R_{\text{deep}} - \beta C_{\text{serverless}}. \quad (5)$$

Esta estructura es de tipo *matching pennies*: el Defensor prefiere D_V ante A_L (bloqueo barato) pero D_A ante A_M

(evita la brecha), mientras que el Atacante prefiere A_M ante D_V pero A_L ante D_A . En consecuencia *no existe equilibrio en estrategias puras* y la solución es un equilibrio completamente mixto [2].

3.4 El principio de indiferencia: balanceando Joules y seguridad

En un juego bimatriz 2×2 sin equilibrio puro, el equilibrio mixto se obtiene mediante el *principio de indiferencia*: cada jugador aleatoriza de modo que su rival quede *indiferente* entre sus dos acciones. Analicemos la condición de indiferencia del *Defensor*, pues es la que revela cómo se equilibran los Joules y la seguridad. El Atacante elige q tal que la utilidad esperada del Defensor sea igual jugando D_V o D_A :

$$\underbrace{q A_{D_V, A_L} + (1 - q) A_{D_V, A_M}}_{\mathbb{E}[U_D | D_V]} = \underbrace{A_{D_A, \cdot}}_{\mathbb{E}[U_D | D_A]}. \quad (6)$$

Sustituyendo (1) y reordenando, la igualdad (6) equivale a

$$\underbrace{\beta (C_{\text{serverless}} - C_{\text{edge}})}_{\text{sobrecosto energético de inspeccionar}} = \underbrace{\gamma (1 - q) C_{\text{damage}} + \Delta R}_{\text{daño evitado + ganancia de detección}}, \quad (7)$$

donde $\Delta R = \alpha (R_{\text{deep}} - q R_{\text{edge}})$. La ecuación (7) es el núcleo analítico de SGE: en el equilibrio, el **sobrecosto energético** de activar la capa Serverless (lado izquierdo, $\beta \Delta C$ Joules) queda *exactamente compensado* por el **daño esperado que se evita** más la ganancia por detección (lado derecho). El Defensor no “sobreinspecciona” (gastaría Joules sin retorno de seguridad) ni “sub-inspecciona” (aceptaría daño evitable): el equilibrio de Nash fija el punto exacto en que el Joule marginal invertido iguala a la seguridad marginal obtenida.

Simétricamente, la probabilidad p con que el Defensor juega D_V se fija para hacer indiferente al *Atacante*:

$$p = \frac{B_{D_A, A_M} - B_{D_A, A_L}}{(B_{D_V, A_L} - B_{D_A, A_L}) - (B_{D_V, A_M} - B_{D_A, A_M})}, \quad (8)$$

siendo $\mathbf{B}$ la matriz de pagos del Atacante. Es notable que p dependa de la matriz del Atacante: es el *incentivo del adversario* —no la mera aversión al riesgo del defensor— lo que determina la tasa óptima de aleatorización, un resultado consistente con la literatura de juegos de seguridad [1, 2].

Ejemplo resuelto. Para una petición con alta anomalía de payload ($s = 0.712$), las matrices son

$$\mathbf{A} = \begin{pmatrix} 9.00 & -106.73 \\ -3.00 & -3.00 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} -5.00 & 131.04 \\ -5.00 & -35.00 \end{pmatrix}.$$

Aplicando (8) y su análoga para q se obtiene el equilibrio mixto $p = 0.181$ y $q = 0.896$: el Defensor **escala a Serverless con probabilidad** $1 - p = 0.819$,

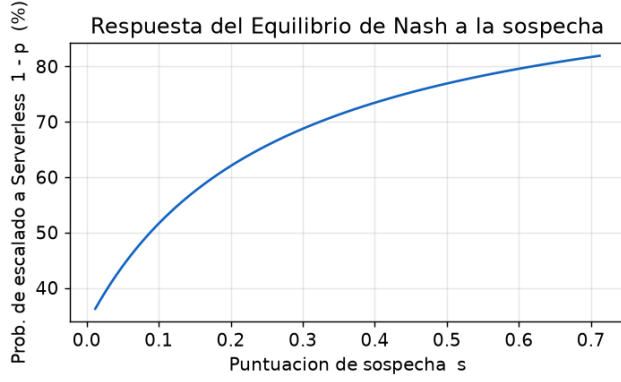


Figura 1: Respuesta del equilibrio de Nash: probabilidad de escalar a Serverless ($1 - p$) frente a la puntuación de sospecha s . El escalado es despreciable para tráfico benigno y crece con la anomalía del payload.

concentrando la energía en el tráfico de alto riesgo. La Figura 1 muestra cómo la probabilidad de escalado crece monótonamente con la sospecha, materializando la regla analítica de (7).

4 Arquitectura del Sistema

SGE simula dos capas de infraestructura cloud con costos energéticos asimétricos.

Capa Edge (filtro ligero). Función de paso rápido con costo $C_{\text{edge}} = 1$. Extrae métricas baratas de la petición —frecuencia de la IP de origen, tamaño del payload y anomalía del *User-Agent*— y ejecuta el *cerebro matemático* (el motor de Nash de la Sección 3). Aplica además *rate-limiting* que frena los bots ruidosos (alta frecuencia de IP y firma de agente anómala) sin invocar la capa costosa. Los ataques avanzados imitan tráfico legítimo y *no* son detenidos aquí.

Capa Serverless (inspección profunda). Función efímera que realiza sanitización exhaustiva y validación criptográfica del payload. Permanece inactiva salvo que el modelo decida activarla. Su costo de ejecución es $C_{\text{exec}} = 15$ y, tras un periodo de inactividad, incurre en *cold start* con costo $C_{\text{cold}} = 25$ [5]. Al invocarse, detecta de forma fiable cualquier clase de ataque. La relación de costos $C_{\text{edge}} : C_{\text{exec}} : C_{\text{cold}} = 1 : 15 : 25$ refleja el elevado sobrecoste energético de la inspección profunda, cuya cuantificación en Joules es central para el ecodiseño de software [3, 4].

Tabla 1: Comparación de enfoques (semilla 7, $n = 2,000$). Energía en Joules simulados.

Enfoque	Mit. global	Mit. avanz.	Energía
Tradicional (100%)	100%	100%	30,010
Adaptativo (Nash)	94.38%	78.95%	4,225
Edge-only (0%)	75.53%	0%	2,000

5 Metodología y Resultados

5.1 Diseño experimental

Simulamos un flujo constante de $n = 2,000$ peticiones (semilla 7) con la mezcla 80% usuarios legítimos, 15% bots ruidosos (A_L) y 5% hackers dirigidos (A_M). Los legítimos usan IP diversa, agente de navegador y payload pequeño y limpio; los bots concentran pocas IPs (alta frecuencia) con agentes anómalos; los avanzados usan agente cuasi-normal y payload voluminoso con *tokens* de inyección. Comparamos SGE (**Adaptativo**) contra dos líneas base: **Tradicional** (inspección estática del 100% en Serverless) y **Edge-only** (nunca escala). Las métricas son la tasa de mitigación (global y de ataques avanzados) y la energía total en Joules simulados [3]. La implementación es modular y su calidad se verificó con el analizador estático *Ruff* y con una batería de 13 pruebas en *Pytest* (motor de Nash, condición de indiferencia y simulación).

5.2 Resultados principales

La Tabla 1 resume la comparación. SGE mitiga el **94.38%** del total de ataques —bloqueando el 100% de los ataques ligeros (313/313) y el 78.95% de los avanzados (90/114) en esta corrida, con un rango típico de ~ 79 –88% a través de múltiples semillas— mientras reduce el consumo energético en un **85.9%** (de 30,010 a 4,225 Joules) respecto a la inspección estática total. El enfoque Edge-only minimiza la energía pero deja pasar la totalidad de los ataques avanzados (0% de mitigación), confirmando que la capa profunda es imprescindible para la fracción sigilosa.

5.3 Análisis: por qué el modelo es tan eficiente

La clave del ahorro reside en la *distribución del costo por capa*. En la corrida titular, de las 2,000 peticiones solo 107 (5.35%) se escalaron a la capa Serverless; las 1,893 restantes se resolvieron en el Edge al costo mínimo $C_{\text{edge}} = 1$. Esto ocurre por dos mecanismos complementarios:

1. El 80% de **tráfico legítimo** produce una sospecha s por debajo del umbral de vía rápida, de modo que

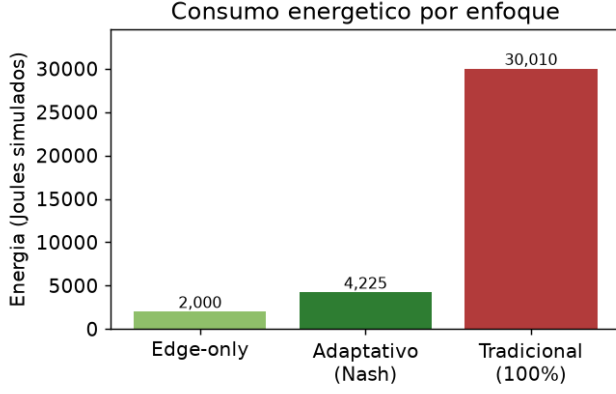


Figura 2: Consumo energético por enfoque. El Adaptativo se sitúa cerca del piso del Edge-only y muy por debajo de la inspección estática Tradicional.

nunca activa el motor completo ni la capa Serverless: se sirve a costo 1.

2. El 15% de **bots ruidosos** se neutraliza en el Edge por *rate-limiting* (firma de agente y alta frecuencia de IP), también a costo 1, sin pagar el sobre costo de la inspección profunda.

En consecuencia, la energía total del Adaptativo se descompone en 2,000 Joules del Edge (uno por petición) más 2,225 Joules de las 107 invocaciones Serverless (62 con *cold start* a 25 y 45 tibias a 15), totalizando 4,225 Joules. La línea Tradicional, en cambio, paga entre 15 y 25 Joules por *cada* petición, alcanzando 30,010. El equilibrio de Nash reserva analíticamente el gasto costoso para el $\sim 5\%$ de tráfico donde la inspección profunda aporta valor real, materializando el principio de ecodiseño de reducir el sobreprocesamiento innecesario [4, 3].

5.4 Frontera de compromiso

La Figura 3 traza la frontera seguridad-energía al variar el incentivo de brecha asumido para el adversario (G_0 , proxy del valor de la API). A mayor incentivo, el equilibrio prescribe más escalado y por tanto mayor mitigación avanzada a costa de más energía, aproximándose asintóticamente al punto Tradicional. Aún en su extremo más seguro ($\sim 92\%$ de mitigación avanzada) el Adaptativo consume del orden de 4,260 Joules, un orden de magnitud por debajo de los 30,010 del enfoque estático.

5.5 Justificación de la reducción de la huella de carbono

El ahorro energético de SGE se traduce de forma directa en una reducción de emisiones de CO_2 . La huella de carbono de una carga de cómputo se modela habitualmente como el producto de la energía consumida por la

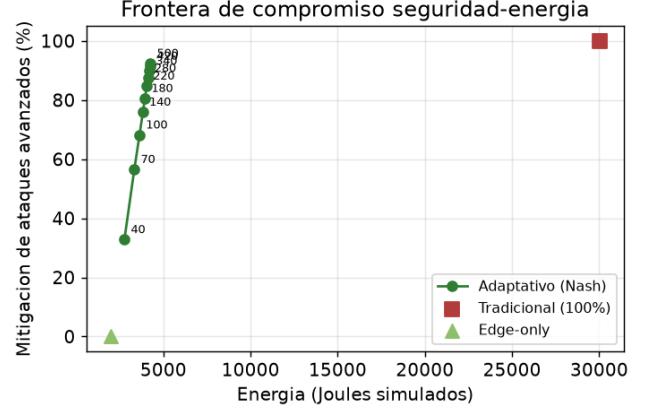


Figura 3: Frontera de Pareto seguridad-energía del enfoque Adaptativo al variar el incentivo de brecha G_0 (etiquetas junto a cada punto), con los puntos de referencia Tradicional y Edge-only.

intensidad de carbono de la electricidad que la alimenta [6, 4]:

$$E_{\text{CO}_2} = W \cdot \text{PUE} \cdot I_{\text{grid}}, \quad (9)$$

donde W es la energía de cómputo (kWh), PUE (*Power Usage Effectiveness*) captura el sobre costo del centro de datos —refrigeración, distribución— (típicamente 1.4–1.6 [6]) e I_{grid} es la intensidad de carbono de la red eléctrica ($\text{gCO}_2\text{eq/kWh}$). Para una misma infraestructura, PUE e I_{grid} son factores constantes, por lo que la reducción *relativa* de emisiones coincide exactamente con la reducción *relativa* de energía:

$$\frac{\Delta E_{\text{CO}_2}}{E_{\text{CO}_2}} = \frac{\Delta W}{W} = 85.9\%. \quad (10)$$

Es decir, al recortar en un 85.9% la energía de la inspección de amenazas (Sección 5), SGE recorta en la *misma proporción* las emisiones de carbono atribuibles a dicha carga. Este ahorro se sustenta en tres mecanismos complementarios:

1. **Menos cómputo total.** El 94.65% de las peticiones se resuelve en el Edge al costo mínimo $C_{\text{edge}} = 1$, evitando los 15–25 Joules de la inspección profunda. Menos ciclos de CPU implican menos energía y, por (9), menos CO_2 .
2. **Especialización hacia el borde.** Al no escalar a la función Serverless, esas peticiones no incurrn en el sobre costo de PUE del centro de datos centralizado ni en la energía de transmisión de red asociada, magnitudes que la ecuación (9) amplifica sobre el gasto bruto de cómputo [4, 6].
3. **Menos arranques en frío.** Concentrar las invocaciones en el tráfico realmente sospechoso reduce los *cold starts* (los más costosos, $C_{\text{cold}} = 25$), recortando el pico energético por evento [5].

Estimación ilustrativa a escala. Para dimensionar el impacto, proyectamos el ahorro simulado a un despliegue real. Anclando de forma conservadora el costo de una inspección profunda tibia en $e_{\text{exec}} = 1$ J de energía de cómputo (equivalente a $C_{\text{exec}} = 15$ unidades del modelo), el ahorro medio es de ≈ 0.86 J por petición. Para una API con $n = 10^9$ peticiones mensuales, con $\text{PUE} = 1.5$ e $I_{\text{grid}} = 0.475$ kgCO₂/kWh (media mundial de la red [6]), esto equivale a evitar

$$\Delta E_{\text{CO}_2} \approx \frac{0.86 \text{ J} \times 10^9}{3.6 \times 10^6 \text{ J/kWh}} \cdot 1.5 \cdot 0.475 \approx 170 \frac{\text{kg CO}_2}{\text{mes}},$$

esto es, del orden de **2 toneladas de CO₂ al año** —comparable a conducir un automóvil promedio unos 8,000 km— respecto a la inspección estática total, y únicamente por trasladar la seguridad de un modelo de inspección *invariante* a uno *adaptativo guiado por Nash*. Aunque las magnitudes absolutas dependen de las hipótesis (energía por inspección, PUE e I_{grid}), la reducción *proporcional* del 85.9% —resultado central de (10)— es independiente de ellas. En síntesis, SGE materializa el principio de ecodiseño de eliminar el sobreprocesamiento innecesario [4, 3], convirtiendo un ahorro de Joules en un ahorro directo y cuantificable de huella de carbono.

6 Discusión

Los resultados evidencian que aleatorizar la inspección según el equilibrio de Nash captura la mayor parte del valor de seguridad de la inspección exhaustiva con una fracción de su costo energético, alineando la ciberseguridad con los objetivos del cómputo verde [4, 3]. La especialización por capa —Edge gratuito para bots ruidosos, Serverless reservado para anomalías de payload— permite que el 94.65% de las peticiones jamás toquen la capa costosa.

Limitaciones. (i) El modelo de detección Serverless es idealizado (detección perfecta al inspeccionar); incorporar falsos positivos/negativos es trabajo futuro. (ii) Cada petición se trata como una etapa independiente; un verdadero juego estocástico multi-etapa con estado (reputación de IP, calentamiento de la función) permitiría políticas dependientes de la historia [1]. (iii) La puntuación de sospecha usa heurísticas ligeras; aprenderla de datos reales mejoraría su calibración [2]. (iv) Los costos energéticos son unidades relativas simuladas, no mediciones físicas [3].

7 Conclusiones

Presentamos SGE, una defensa perimetral adaptativa para APIs que formula la decisión de inspección profunda como un juego estocástico no cooperativo con in-

formación imperfecta y la resuelve mediante el equilibrio de Nash en estrategias mixtas. Analíticamente, el principio de indiferencia iguala el Joule marginal invertido con la seguridad marginal obtenida; empíricamente, SGE logra un 94.38% de mitigación global y bloquea el 100% de los ataques ligeros con un 85.9% de ahorro energético —y, en consecuencia, una reducción proporcional del 85.9% en la huella de carbono de la carga de inspección (Sección 5.5)— frente a la inspección estática total. El enfoque demuestra que la teoría de juegos es una herramienta efectiva para conciliar la seguridad con la sostenibilidad. Como trabajo futuro proponemos extender el modelo a un juego estocástico multi-etapa con estado y aprender tanto la función de sospecha como los parámetros de utilidad a partir de trazas de tráfico reales.

Referencias

- [1] T. Alpcan and T. Başar. *Network Security: A Decision and Game-Theoretic Approach*. Cambridge University Press, 2010.
- [2] M. H. Manshaei, Q. Zhu, T. Alpcan, T. Başar, and J.-P. Hubaux. *Game theory meets network security and privacy*. ACM Computing Surveys (CSUR), 45(3):1–39, 2013.
- [3] S. Murugesan and G. R. Gangadharan (eds.). *Harnessing Green IT: Principles and Practices*. Wiley, 2012.
- [4] L. D. Radu. *Green Cloud Computing: An Energy-Efficient Ecodesign for IT*. Energies, 2017.
- [5] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski. *The rise of serverless computing*. Communications of the ACM, 62(12):44–54, 2019.
- [6] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey. *Recalibrating global data center energy-use estimates*. Science, 367(6481):984–986, 2020.